位运算符（基本上用不着，所以我忘了）

基本语法：

位运算符		
运算符	运算	范例
<<	左移	3 << 2 = 12 --> 3*2*2=12
>>	右移	3 >> 1 = 1 --> 3/2=1
>>>	无符号右移	3 >>> 1 = 1 --> 3/2=1
&	与运算	6 & 3 = 2
|	或运算	6 | 3 = 7
^	异或运算	6 ^ 3 = 5
~	取反运算	~6 = -7

注意：无<<<

位运算符的细节	
<<	空位补0，被移除的高位丢弃，空缺位补0。
>>	被移位的二进制最高位是0，右移后，空缺位补0； 最高位是1，空缺位补1。
>>>	被移位二进制最高位无论是0或者是1，空缺位都用0补。
&	二进制位进行&运算，只有1&1时结果是1，否则是0;
|	二进制位进行 | 运算，只有0 | 0时结果是0，否则是1;
^	相同二进制位进行 ^ 运算，结果是0；1^1=0 , 0^0=0 不相同二进制位 ^ 运算结果是1。1^0=1 , 0^1=1
~	正数取反，各二进制码按补码各位取反 负数取反，各二进制码按补码各位取反

位运算符的运算过程都是基于底层二进制补码进行的。

(1)左移：<<

运算规则：在一定范围内，数据每向左移动一位，相当于原数据*2。（正数、负数都适用）

例如：

> 3 << 4 类似于 3 * 2的4次幂 => 3 * 16 => 48

> -3 << 4 类似于 -3*2的4次幂 => -3*16 => -48

（2）右移

运算规则：在一定范围内，数据每向右移动一位，相当于原数据/2。（正数、负数都适用）

注意：**如果不能整除，向下取整**。

例如：

> 69>>4 类似于 69/2的4次 = 69/16 =4

> -69 >> 4 类似于 -69 / 2的四次幂 = -69 / 16 = -5

对于左移<<与右移>>运算符，它们用于进行乘2的n次幂以及除以2的n次幂的运算速度比乘法除法的运算速度快，但也就仅仅快一点点。

使用左移、右移运算符时，需要注意可能字符移动后会超出字符长度，此时就会出现错误

例如：

```
System.out.println(10 << 100);
```

此时输出结果为：

明显结果是错误的，原因就在于二进制补码向左移动了太多位，导致超出了int类型数据存储的长度，导致发生数据错误。

所以**左移、右移运算符是有一定范围的，在使用时不能超出该范围**。

（3）无符号右移：>>>

运算规则：往右移动后，左边出来的位直接补0。（正数、负数都适用）